Министерство образования и науки Российской Федерации

Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина

ЛАБОРАТОРНАЯ РАБОТА №6

“Приложения машинного обучения“

Студент:
Группа РИ-240917 *Холкин Николай*

Преподаватель: *Решетников Кирилл Игоревич*

Екатеринбург
13 мая 2026 г.

1 Docker Compose

Код лабораторной работы доступен в репозитории:

https://github.com/HowardLovekraft/UrFU-MLOps

Код Dockerfile для сборки бэкенда:

```
FROM ghcr.io/astral-sh/uv:debian-slim

WORKDIR /usr/src/Lab_6

COPY pyproject.toml ./
RUN uv sync
COPY . ./

ENV PYTHONPATH=/usr/src/Lab_6
EXPOSE 8000

CMD ["uv", "run", "src/app.py"]
```

Код compose.yaml - приложения из API модели и БД:

```
services:
  web:
    build: .
    ports:
      - 8000:8000
    restart: always

  db:
    image: postgres:18
    restart: always
    shm_size: 128mb
    environment:
      POSTGRES_PASSWORD: 1337
```

```
➜  ~ curl -X POST 127.0.0.1:8000/predict \
     -H "Content-Type: application/json" \
     -d '{
        "period": "3rd Period",
        "subject": "English",
        "co2_ppm": 663.8,
        "pm25_ugm3": 8.07,
        "temperature_c": 21.9,
        "humidity_pct": 46.0,
        "reaction_time_ms": 216,
        "focus_rating": 7.2,
        "error_rate": 1.68,
        "heart_rate_bpm": 75,
        "cognitive_impairment": 0.0191,
        "air_quality": 2
     }'

{"performance_index":1}%
```

2 Автотесты

Для программы написаны тесты на pytest. Всего 3 теста:

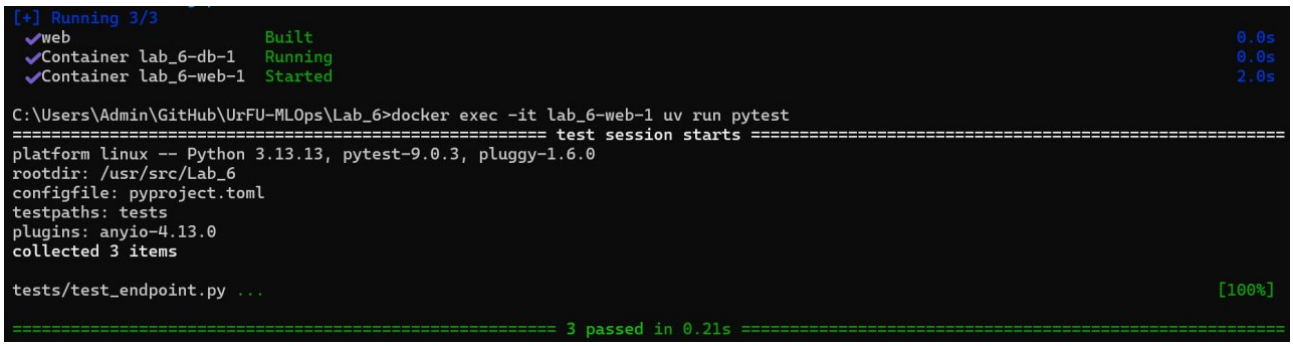
- Корректная работа эндпоинта
- Проверка валидатора запроса к модели
- Проверка базы данных

Код тестов:

```python
from typing import Final

from fastapi import status
from fastapi.testclient import TestClient
from httpx import Response

from src.db import get_record
from src.schemas import RecordSchema, PredictionSchema


ENDPOINT_URL: Final = '/predict'


def test_valid_request(
    client: TestClient,
    valid_request: RecordSchema,
    valid_response: PredictionSchema
) -> None:
    response: Response =  client.post(ENDPOINT_URL, json=valid_request.model_dump())
    assert response.status_code == 200
    assert response.json() == valid_response.model_dump()


def test_invalid_enum_in_request(
    client: TestClient,
    request_with_invalid_enum: dict
) -> None:
    response: Response = client.post(ENDPOINT_URL, json=request_with_invalid_enum)
    assert response.status_code == status.HTTP_422_UNPROCESSABLE_CONTENT


def test_db(
    client: TestClient,
    valid_request_2: RecordSchema
) -> None:
    assert get_record(valid_request_2) is None
    client.post(ENDPOINT_URL, json=valid_request_2.model_dump())
    assert isinstance(get_record(valid_request_2), int)
```

3 PostgreSQL

В приложении используется база данных Postgres для хранения запросов пользователей.



Поддерживается 2 операции: добавление записи и получение значения таргета по записи.

Код запросов к БД:

```python
import psycopg

from src.schemas import RecordSchema


def create_table():
    with (
        psycopg.connect("host=db dbname=postgres user=postgres password=1337") as
        ↪  conn,
        conn.cursor() as cur
    ):
        cur.execute("""
            CREATE TABLE IF NOT EXISTS predictions (
                id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                period TEXT,
                subject TEXT,
                co2_ppm DOUBLE PRECISION,
                pm25_ugm3 DOUBLE PRECISION,
                temperature_c DOUBLE PRECISION,
                humidity_pct DOUBLE PRECISION,
                reaction_time_ms INTEGER,
                focus_rating DOUBLE PRECISION,
                error_rate DOUBLE PRECISION,
                heart_rate_bpm INTEGER,
                cognitive_impairment DOUBLE PRECISION,
                air_quality INTEGER,
                performance_index INTEGER
            )
        """)


def get_record(record: RecordSchema):
    with (
        psycopg.connect("host=db dbname=postgres user=postgres password=1337") as
        ↪  conn,
        conn.cursor() as cur
    ):
        cur.execute("""
            SELECT performance_index
```

```
              FROM predictions
             WHERE period = %s
               AND subject = %s
               AND co2_ppm = %s
               AND pm25_ugm3 = %s
               AND temperature_c = %s
               AND humidity_pct = %s
               AND reaction_time_ms = %s
               AND focus_rating = %s
               AND error_rate = %s
               AND heart_rate_bpm = %s
               AND cognitive_impairment = %s
               AND air_quality = %s
        """, tuple(record.model_dump().values()))
        result: tuple[int] | None = cur.fetchone()
        if result is None:
            return None
        return result[0]


def add_record(record: RecordSchema, performance_index: int) -> None:
    with (
        psycopg.connect("host=db dbname=postgres user=postgres password=1337") as
        ↪  conn,
        conn.cursor() as cur
    ):
        cur.execute(
            """
            INSERT INTO predictions
            VALUES (DEFAULT, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            """,
            tuple(record.model_dump().values()) + (performance_index,)
        )
        conn.commit()
```